

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

A CASE STUDY REPORT ON

Argus: Cross-Database Query Engine with an ML-Based Optimizer

---

Submitted for the course

BCSE302L: Database Systems

Submitted by

Divyanshu Singh

Registration No. 24BCT0101

Vellore Institute of Technology

October 2026

INDEX

Contents	Page No.
Abstract	1
1. Introduction	1
1.1 The multi-database problem	1
1.2 Problem statement and objectives	2
2. Motivation (Literature Survey)	2
2.1 Approximate query processing	2
2.2 Sampling theory and joins	3
2.3 Federated and polystore systems	3
2.4 Learned performance models	3
2.5 Research gap	3
3. Novelty in the present work	4
3.1 PAFS: Pilot-guided Adaptive Federated Sampling	5
3.2 ML-based optimizer that chooses between exact and approximate	5
3.3 A source-aware optimiser that unites SQL and NoSQL	6
3.4 Transparent, verifiable answers	6
3.5 Argus compared with conventional approaches	6
3.6 Summary of contributions	7
4. Implementation of work	7
4.1 Architecture	7
4.2 Connecting SQL and NoSQL	8
4.3 Query optimisation	8
4.4 Approximate query engine	9
4.5 ML cost model and Auto mode	9
4.6 Database concepts applied	10
4.7 Interface and testing	10
5. Results and discussion	11
5.1 Experimental setup	11
5.2 Where approximation pays off	11
5.3 One query across MySQL, PostgreSQL and MongoDB	12
5.4 ML optimizer in action	13
5.5 Accuracy and reliability	14
6. Conclusion and future scope	14
References	15

ABSTRACT

Most organisations do not keep their data in one place. Orders sit in MySQL, reference data in PostgreSQL and semi-structured notes in MongoDB, and the moment an analyst needs a single number that touches all three, every relevant row has to be pulled across the network into one engine. Argus takes a different route. It connects SQL and NoSQL databases through a DuckDB federation layer, optimises each query by pushing work down to the source wherever possible, and answers aggregates with PAFS (Pilot-guided Adaptive Federated Sampling), a new sampling algorithm that works from samples stored locally and attaches a 95% confidence interval to every value. A gradient-boosted machine-learning model, trained on 886 measured runs over 26 datasets, predicts how long both ways will take before anything runs, and in Auto mode it runs only the faster one, so an estimate is never used where it would be slower. On a query joining MySQL, PostgreSQL and MongoDB over 10.6 million rows, the exact answer took 241 s and the Argus estimate 28 ms, with a mean error of 0.87%. On datasets the model had never seen, it chose the faster path for 97% of queries.

Keywords: SQL and NoSQL integration, federated query processing, query optimisation, approximate query processing, confidence intervals, learned cost model

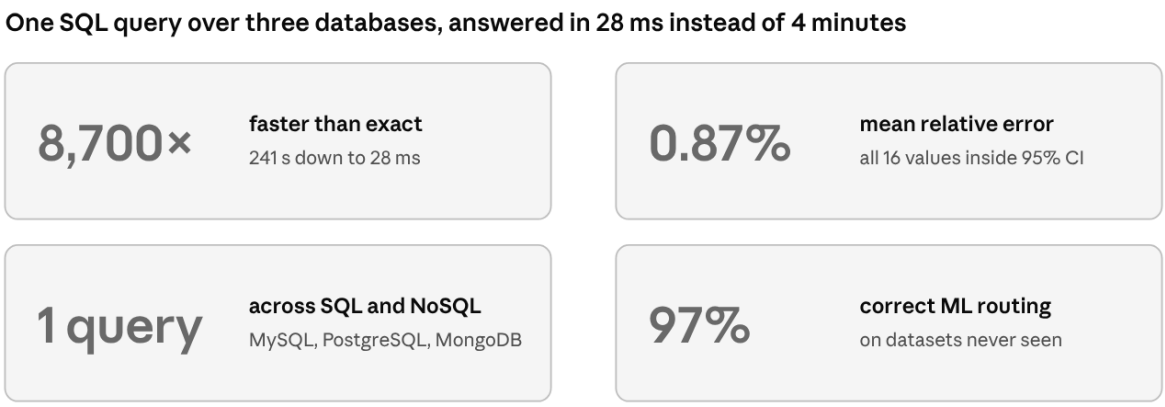


Figure 1. Key results at a glance

1. INTRODUCTION

1.1 The multi-database problem

A typical organisation today runs several databases side by side. Transactions go into MySQL or PostgreSQL because they need joins, constraints and ACID guarantees, while catalogues, logs and user notes go into MongoDB because their structure keeps changing. Each choice is sensible on its own. The trouble starts when someone asks a question that spans them, for instance the number of sales per customer segment when the segment is stored in a document database.

Answering such a question usually means one of three things. The data can be copied every night into a warehouse, which costs storage and leaves the copy out of date. A developer can write glue

code that queries each database separately and joins the results inside the application, which is slow to build and easy to get wrong. Or a federated engine can read every source in place, which is convenient but still drags every qualifying row across the network before anything can be computed. In our measurements, the last option needed about a minute for an aggregate over a 10.6-million-row MySQL table, and four minutes for a join of that table with tables in PostgreSQL and MongoDB.

Many of these questions do not need every digit, though. A dashboard figure or an exploratory query is just as useful at ±1% if it arrives in milliseconds and its error is stated. This is the idea behind approximate query processing (AQP), and this project applies it to the multi-database setting.

1.2 Problem statement and objectives

The problem addressed is the following: given a SQL aggregate query whose tables live in different relational and document databases, return an answer with a known error, in a time that depends neither on the size of the remote tables nor on network latency, and decide before running whether approximation is worthwhile at all. The objectives were (i) to run one SQL query over PostgreSQL, MySQL, MongoDB and local files, joins included; (ii) to optimise each query so that as much work as possible happens inside the source database; (iii) to report a 95% confidence interval for every estimated value; (iv) to make estimates on remote data independent of the network; and (v) to let a machine-learning model choose between the exact and the approximate answer, so that the user never pays for an estimate that would be slower.

Section 2 surveys related work, Section 3 presents the novelty of this project, including the PAFS algorithm and a comparison with conventional approaches, Section 4 describes the implementation, Section 5 presents the results, and Section 6 concludes.

2. MOTIVATION (LITERATURE SURVEY)

2.1 Approximate query processing

Online aggregation showed that an aggregate can be answered from tuples read in random order, with a confidence interval that narrows as more data is read, and ripple joins extended the idea to joins. Aqua answered queries from precomputed samples and, for joins, from join synopses, in which a sample of the fact table is joined in advance with its dimension tables. BlinkDB built stratified samples so that rare groups stay represented, Quickr inserted samplers into query plans at run time, and VerdictDB offered AQP as middleware over an unmodified database. All of these assume that the data sits in one engine or cluster. Chaudhuri, Ding and Kandula added an important warning: no AQP technique suits every workload, so a system must also know when not to approximate. Sketches such as HyperLogLog cover aggregates like COUNT(DISTINCT) that sampling handles poorly.

2.2 Sampling theory and joins

The Horvitz-Thompson estimator makes COUNT and SUM unbiased under any probability sampling design, and AVG, a ratio of two totals, gets its variance by linearisation. Cochran shows how to size a sample from a pilot and how large a sample skewed data needs. Inside a database, block sampling is far cheaper than row sampling, but rows in one block are correlated, so the block must be treated as the sampling unit. Joins are harder still, because the join of two independent samples is not a sample of the join. Join synopses avoid this by sampling only the fact table, while wander join relies on index-based random walks that are not available when the inputs sit on different remote servers.

2.3 Federated and polystore systems

Garlic introduced wrappers through which a central optimiser pushes work to each source, and BigDAWG extended federation to stores with different data models. Presto and Apache Calcite are widely used federated engines with relational and NoSQL connectors, and DuckDB attaches PostgreSQL and MySQL directly. All of them return exact answers only, so an aggregate over a large remote table can never be faster than moving its rows.

2.4 Learned performance models

Machine learning has been used to predict query run time from plan features and from the plan structure itself, and to estimate join cardinalities; gradient-boosted trees remain a strong baseline for tabular features. These models price a single plan. To our knowledge, none chooses between an exact and an approximate answer before the query runs.

2.5 Research gap

Table 1. Comparison of related systems

System	Year	Spans several databases	Document stores	Error bound per value	Decides exact or approximate
Online aggregation	1997	No	No	Yes	No (user stops it)
Aqua, join synopses	1999	No	No	Yes	No
BlinkDB	2013	No (one cluster)	No	Yes	Partly
VerdictDB	2018	No (one backend)	No	Yes	No
BigDAWG	2015	Yes	Yes	No	No
Presto	2019	Yes	Yes	No	No
DuckDB scanners	2019	Yes (SQL only)	No	No	No
Argus (this work)	2026	Yes	Yes (MongoDB)	Yes	Yes (ML model)

The table shows the gap clearly. AQP systems give error bounds but stay inside one engine, federated systems span many databases but compute exact answers only, document stores are

largely ignored by AQP research, and no system decides automatically whether approximating is worth it. Argus is designed to close all of these gaps at once.

3. NOVELTY IN THE PRESENT WORK

Argus introduces a new way of answering analytical queries across heterogeneous databases. Instead of moving all qualifying rows to one engine, as every federated system in Table 1 does, it combines three ideas into one pipeline: a source-aware optimiser that connects SQL and NoSQL databases, a sampling algorithm designed for data that lives on remote servers, and a machine-learning model that decides for each query whether an exact or an approximate answer is the faster route. The sampling algorithm, which we call PAFS (Pilot-guided Adaptive Federated Sampling), is the core of the work. Measured against the approaches used in industry today, it answered the same questions thousands of times faster while keeping the error below one percent (Section 3.5).

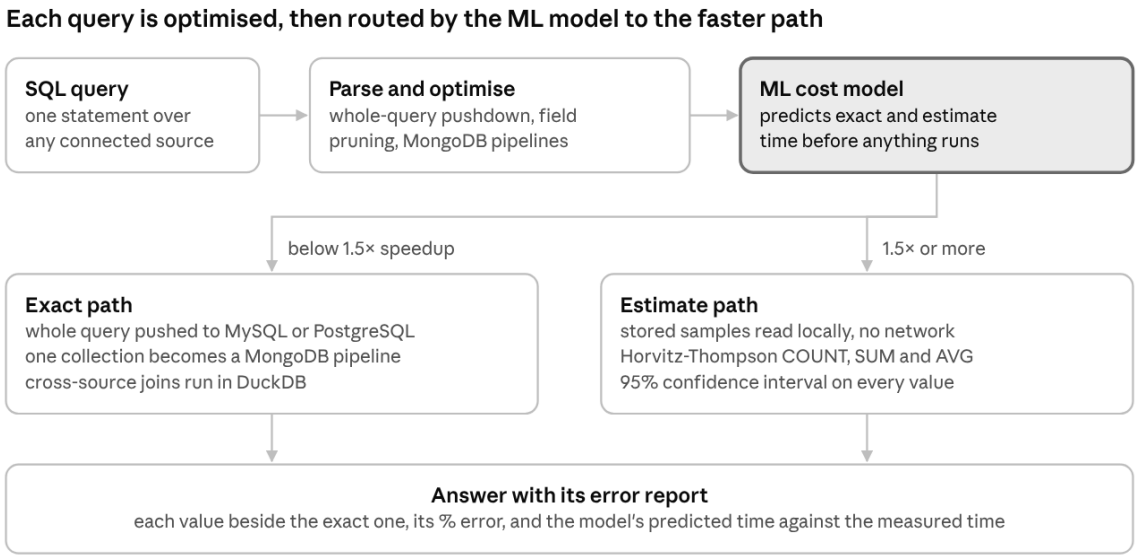


Figure 2. How Argus optimises and routes a query

Table 2. Multi-database problems and how Argus addresses them

Problem	Usual workaround	Argus
Data split across MySQL, PostgreSQL and MongoDB	Nightly ETL into a warehouse, or glue code in the application	One SQL query names tables from any source, e.g. `rfam.full_region` joined with `mongo.family_notes`
Every database speaks its own language	Hand-written dialects and MongoDB pipelines	The query is translated automatically; one-collection queries become aggregation pipelines
Large remote tables make every aggregate slow	Wait, or pre-aggregate by hand	Estimate from a locally stored sample: 241 s became 28 ms on the three-database join
Sampling on the remote server saves nothing	No general workaround	Sample drawn once and kept on disk, so the network leaves the estimate path

Problem	Usual workaround	Argus
On small data an estimate can be slower than the real query	The user has to guess	The ML model predicts both times and Auto mode runs only the faster one
Nobody knows how wrong an estimate is	Trust it, or re-run exactly	A 95% interval and % error on every value, with the model's prediction beside the measured time

3.1 PAFS: Pilot-guided Adaptive Federated Sampling

Classical sampling methods assume that the data can be scanned cheaply in random order. On a remote server this assumption breaks down: MySQL has no sampling clause at all, and server-side sampling in PostgreSQL still reads large parts of the table. PAFS is designed around this constraint. It samples only the largest table of a query, keeps that sample on local disk, and sizes it from a cheap pilot so that the final answer meets the user's error margin with as few rows as possible. Algorithm 1 gives the complete procedure, including the routing decision made by the ML model.

```
Algorithm 1. Argus query processing with PAFS

Input : SQL query Q, error margin e, time budget T
Output: exact answer, or estimates with 95% confidence intervals

 1  parse Q; map every table to its source (MySQL, PostgreSQL, MongoDB, local)
 2  if all tables are in one source then rewrite Q for that source
 3  x <- query features + data location + latency + pilot statistics
 4  t_exact, t_est <- ML cost model (two gradient-boosted regressors)(x)
 5  if t_exact / t_est < 1.5 then return EXACT(Q)      // not worth it
 6  S <- largest table of Q (its stored local sample, if present)
 7  draw a pilot of S at fraction f_p; w_p <- widest relative half-width
 8  f* <- 1 / (1 + (e / w_p)^2 * (1 - f_p) / f_p)
 9  f* <- max(f*, Cochran minimum for skewed data)
10  if f* > 0.5 then return EXACT(Q)                   // saves little
11  repeat
12      sample S at f*; join with the other tables read in full
13      Horvitz-Thompson estimates + 95% intervals for every group
14      enlarge f* along the sampling ladder
15  until all intervals are within e or elapsed time > T
16  return estimates with their intervals
```

Three properties make PAFS stand out. It is unbiased across database boundaries, because every row of the join result descends from exactly one sampled row. It is self-sizing, because the pilot tells it how many rows the requested margin needs, so it never reads more than necessary. And it is network-independent, because once a sample is stored, no estimate touches the remote server again.

3.2 ML-based optimizer that chooses between exact and approximate

Earlier learned cost models predict the cost of a single plan. The optimizer in Argus goes one step further: it compares two competing strategies, exact and approximate, using features of the query, the location of the data and a pilot sample that costs only a few milliseconds. In Auto mode the backend runs only the strategy the model selects. On a local table of one million rows it chose the exact query (3.3 ms); on the remote MySQL table and on the three-database join it chose the

estimate (2.0 ms and 14 ms). Approximation is therefore applied exactly where it pays off, and the user never waits for an estimate that would be slower than the real query.

3.3 A source-aware optimiser that unites SQL and NoSQL

Tables from PostgreSQL, MySQL, MongoDB and local files can be joined in a single SQL statement. MongoDB documents are flattened into relational columns on the fly, so a nested field such as `curation.status` can be filtered like any SQL column. Before any data is fetched, Argus rewrites the query so that as much work as possible happens inside the source: a query on one database is sent there whole in its own dialect, a query on one collection becomes a MongoDB aggregation pipeline, and in a cross-source join only the referenced fields are fetched, with filters applied at the source.

3.4 Transparent, verifiable answers

Every estimate is shown next to the exact value, its 95% confidence interval and its percentage error, and the model's predicted running times are shown next to the measured ones. The user can verify both the answer and the decision that produced it, which none of the federated engines reviewed in Section 2 offers.

3.5 Argus compared with conventional approaches

Table 3 compares Argus with the techniques commonly used in industry on the same data. Every figure was measured in this project.

Table 3. Argus versus conventional approaches on the same workloads

Task	Conventional approach	Result	Argus	Result	Improvement
Aggregate join over MySQL, PostgreSQL and MongoDB (10.6M rows)	Federated exact execution (as in Presto or DuckDB)	241 s	PAFS on a stored sample	28 ms, 0.87% error	8,700× faster
Group-by over a 10.6M-row MySQL table	Exact query pushed into MySQL	59.4 s	PAFS on a stored sample	10.6 ms, 0.51% error	5,600× faster
Sampling a remote MySQL table	`RAND() < 0.01` on the server	Slower than the exact query (full scan)	Sample drawn once, stored locally	Built in 11 s, then milliseconds	No repeated scans
Sampling a 61M-row PostgreSQL table	`TABLESAMPLE SYSTEM (1)`	466 s	Stored sample	Milliseconds per query	Orders of magnitude
Confidence intervals on skewed data	Single fixed-size sample	About 60% coverage	Pilot-sized sample with Cochran minimum	About 95% coverage	Reliable intervals
MongoDB join estimate	Fetch whole documents	16.2 s	Field pruning	0.64 s	25× faster

Task	Conventional approach	Result	Argus	Result	Improvement
Choosing a strategy for 317 queries	Always run the exact query	286.08 s	ML routing (Auto mode)	10.19 s	28× less time
Choosing a strategy for 317 queries	Always use an estimate	28.22 s	ML routing (Auto mode)	10.19 s	2.8× less time

The pattern is consistent. Wherever the conventional approach has to move or scan the full remote table, Argus answers in milliseconds with an error below one percent, and wherever an estimate would not help, the ML optimizer avoids it.

3.6 Summary of contributions

1. PAFS, a sampling algorithm for federated data that is unbiased across database boundaries, sizes itself from a pilot and never waits on the network once a sample is stored.
2. An ML-based optimizer that predicts the running time of both the exact and the approximate plan and executes only the faster one.
3. A source-aware query optimiser that joins relational and document databases in one SQL statement and pushes work into each source.
4. Measured speedups of up to 8,700× over federated exact execution, with a mean error of 0.87% and correct routing for 97% of queries on unseen data.
5. A complete working system, with a web workbench, 61 automated tests and live connections to public MySQL and PostgreSQL servers and a MongoDB Atlas cluster.

4. IMPLEMENTATION OF WORK

4.1 Architecture

Argus is built in four layers (Figure 3). A React and TypeScript workbench sends SQL to a FastAPI backend written in Python 3.12, which holds the exact engine, the approximate engine and the ML optimizer. All three share one DuckDB 1.5 connection that acts as the hub of the federation: it attaches the relational databases, pulls in the parts of MongoDB collections a query needs through PyMongo, and keeps samples of remote tables in its own schema, `aq_samples`. Queries are parsed and translated with sqlglot, and the model is built with scikit-learn.

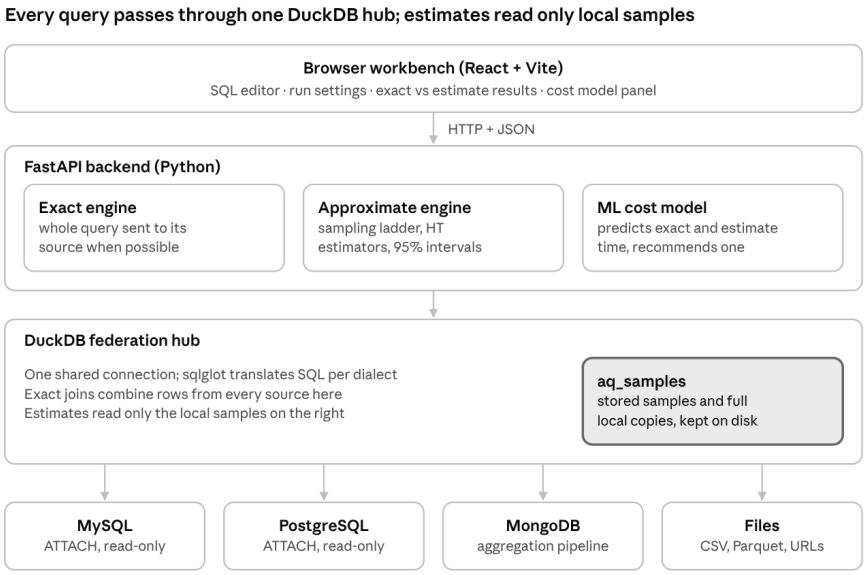


Figure 3. System architecture of Argus

4.2 Connecting SQL and NoSQL

PostgreSQL and MySQL are attached read-only through DuckDB's scanner extensions, so their tables can be addressed as `<connection>.<table>`, for example `rfam.full_region`. MongoDB has no SQL interface, so Argus samples up to 1,000 documents of each collection to infer a schema and flattens nested fields into columns (`curation.status` becomes `curation_status`). From the user's point of view, a MongoDB collection then behaves like any other table and can be filtered, grouped and joined with relational tables in the same statement. Connections are made from the workbench with a connection string, and the public Rfam and RNAcentral databases can be attached with a single click.

4.3 Query optimisation

The optimiser's job is to move work to where the data is and to move as little data as possible. Table 4 lists the optimisations Argus applies and their measured effect.

Table 4. Query optimisations in Argus

Optimisation	What Argus does	Measured effect
Whole-query pushdown	If all tables live in one database, the query is translated into its dialect and run there	Only the result crosses the network: one row for a 10.6-million-row Rfam aggregate
MongoDB pipeline rewrite	A query over one collection becomes a `$match`, `$group`, `$project` pipeline	Aggregation runs inside MongoDB
Field pruning	In cross-source joins, only the referenced fields are fetched from MongoDB	Two-collection join estimate cut from 16.2 s to 0.64 s
Predicate pushdown	Filters that MongoDB can evaluate are applied in `$match`	Fewer documents transferred
Stored samples	A remote table is sampled once and kept on local disk	Estimates in milliseconds; remote `RAND() < 0.01` still read all 10.6 million rows

Optimisation	What Argus does	Measured effect
Local copies of small tables	Small join partners are copied once	Rfam join estimate cut from 3.4 s to 38 ms
ML routing	The cheaper of exact and approximate execution is chosen before running	Slower estimates are never run in Auto mode

4.4 Approximate query engine

The engine approximates COUNT, SUM and AVG with WHERE, GROUP BY, ORDER BY, LIMIT and inner joins. Samples are Poisson samples in which each unit is kept independently with probability f. Large local tables are sampled in blocks of 2,048 rows, which are then the sampling unit because rows within a block are correlated; smaller tables, MongoDB collections and stored samples are sampled by row. In a join, only the largest table is sampled and the others are read in full, so each joined row derives from exactly one sampled row and the scaled COUNT and SUM stay unbiased. A total is estimated with the Horvitz-Thompson estimator, with its variance taken from the same sample:

$$\hat{Y} = (1 / f) \cdot \Sigma\, y_i \quad \mathrm{Var}(\hat{Y}) = ((1 - f) / f^2) \cdot \Sigma\, y_i^2 \quad (1)$$

where both sums run over the sampled units $i \in S$.

COUNT uses $y = 1$, the 95% interval is the estimate plus or minus 1.96 standard errors, and AVG is the ratio of SUM and COUNT with a linearised variance. To choose the sample size, a small pilot sample at fraction f_p measures the widest relative half-width w_p. Because the half-width shrinks with the square root of $(1 - f)/f$, the fraction needed for a margin ε follows from Equation 2, and the answer comes from a fresh sample of that size.

$$f^* = 1 / [\, 1 + (\varepsilon / w_p)^2 \cdot (1 - f_p) / f_p \,] \quad (2)$$

where f^* is the sampling fraction needed to meet the margin ε.

Skewed data must also yield at least 25 g_1^2 values, g_1 being the skewness. Sampling stops once every interval meets the margin (±20%, ±8% or ±2% for the Fast, Balanced and Accurate profiles), when the time budget of 0.75 s, 1.5 s or 3 s runs out, or when the margin would need more than half of the rows, in which case the exact query is recommended. Samples of remote tables are built once: the 1% sample of `rfam.full_region` (107,219 rows) took 11 s and has been reused ever since.

4.5 ML cost model and Auto mode

The model sees the query before it runs. Its features describe table sizes and locations, joins, groups and filters, stored samples, network latency, pushdown and the accuracy target, plus a pilot sample costing a few milliseconds (its time, interval width and the fraction the target will need). Two gradient-boosted regressors (400 trees, depth 3, learning rate 0.05) predict the logarithm of the exact and the approximate running time, and the estimate is chosen only if its predicted speedup is at least 1.5×.

The model was trained on 886 logged runs, merged into 320 distinct queries over 26 datasets (synthetic tables of 20,000 to 6 million rows, TPC-H, Rfam, RNAcentral and queries run in the app), and evaluated with leave-one-dataset-out cross-validation, so the reported accuracy always

concerns data the model had never seen. In Auto mode, the backend asks the model first and then runs only the chosen strategy; once table sizes and network latency are cached, this decision takes just 6 to 18 ms. Every query run through the app is logged as new training data, so the optimizer keeps improving with use.

4.6 Database concepts applied

Argus puts many core topics of a database systems course into practice. Table 5 maps them to the parts of the system where they appear.

Table 5. Database system concepts applied in Argus

Concept	Where it appears in Argus
Relational and NoSQL data models	MySQL and PostgreSQL tables joined with MongoDB collections; documents flattened into relational columns
Query processing and optimisation	Selection pushdown, projection pushdown (field pruning), whole-query pushdown and join planning
Distributed and federated databases	One DuckDB hub attaches remote servers, so a single query spans three database systems
Query translation	SQL rewritten into the MySQL and PostgreSQL dialects and into MongoDB pipelines (`$match`, `$group`, `$project`)
Cost-based optimisation	The ML model estimates the cost of the exact and the approximate plan before choosing one
Aggregation and grouping	COUNT, SUM and AVG with GROUP BY, estimated with confidence intervals
Indexing and storage	Unique index on `rfam_acc` in MongoDB; DuckDB's columnar storage for samples and local tables
Materialisation and caching	Stored samples and local copies kept as materialised tables in `aq_samples`; a result cache for repeated queries
Metadata and catalogues	Table sizes read from catalogues (`table_rows`, `reltuples`); MongoDB schemas inferred from sampled documents
Security and access control	Remote sources attached read-only; credentials held only in memory

4.7 Interface and testing

The workbench (Figure 4) lists all connected sources, offers an SQL editor with completions, and has four ways to run a query: exact, estimate, both side by side, and Auto, where the ML optimizer decides. After each run it shows both running times, the speedup, every exact value beside its estimate, interval and error, and how the model's prediction compared with the measured time. Further tabs show the SQL or MongoDB pipeline that was actually executed, the query plan as a graph, the sequence of samples the estimate took and the query history, so every step of the processing can be inspected. A list of recommended queries, ranked by predicted speedup, helps users discover where Argus is fastest. The backend is covered by 61 automated tests, including checks of the SQL-to-MongoDB translation against DuckDB on the same documents and of Auto mode's routing.

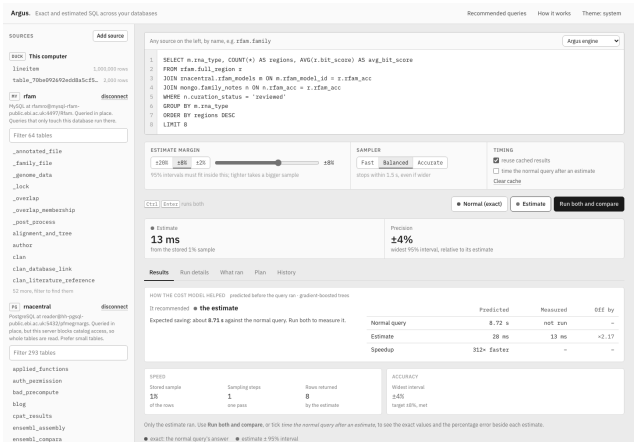


Figure 4. The Argus workbench running the three-database query: connected sources, SQL editor, run settings, and the estimate returned in 13 ms

5. RESULTS AND DISCUSSION

5.1 Experimental setup

All experiments ran on one laptop, so remote timings include the internet round trip. The sources were the public Rfam MySQL database (`full_region`, 10.6 million rows), the public RNAcentral PostgreSQL database, both hosted by EMBL-EBI, and a MongoDB Atlas cluster holding `sample_mflix` and `family_notes`, a collection of 4,227 documents built for this study with real Rfam accessions and generated curation fields. Local tests used TPC-H. Estimates used the Balanced profile (±8%) with stored 1% samples.

5.2 Where approximation pays off

To map out where approximation helps, Figure 5 forces the estimate on every kind of source and compares it with the exact query.

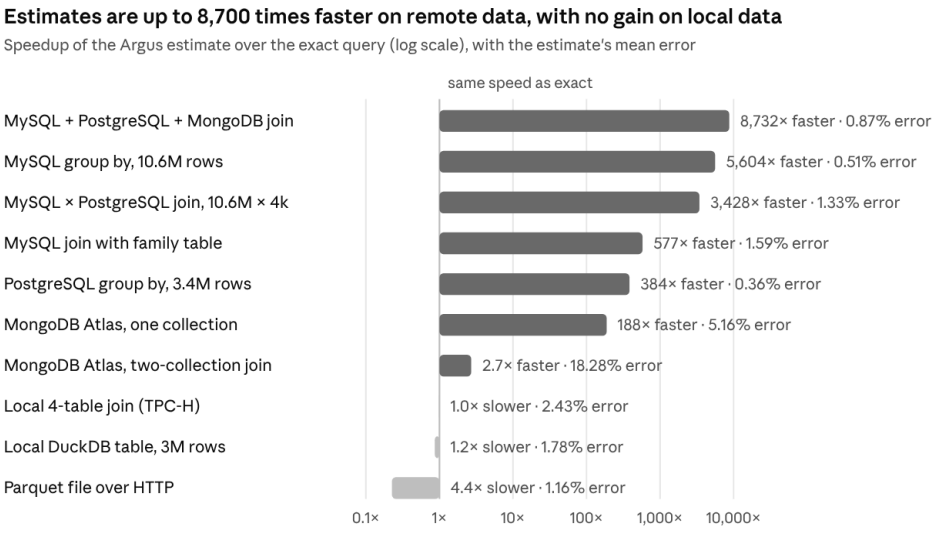


Figure 5. Speedup and mean error of the estimate when it is forced on every source type

On remote databases the estimate was up to 8,700 times faster with a mean error below 6%, and on the large MySQL and PostgreSQL tables the error stayed below 2%. The largest gains come from the three-database join and the 10.6-million-row MySQL table, where the exact query has to move millions of rows across the internet while PAFS reads a 1% sample from local disk. Even the 3.4-million-row PostgreSQL table was answered 384 times faster with an error of only 0.36%, and a single-collection MongoDB Atlas aggregate was 188 times faster.

On local data, where DuckDB already answers exact aggregates in milliseconds, sampling brings no extra speed. This is exactly the case the ML optimizer recognises: in Auto mode such queries are simply run exactly, so the user always gets the faster of the two (Section 5.4).

5.3 One query across MySQL, PostgreSQL and MongoDB

The headline experiment joins all three systems on the Rfam family accession:

```
SELECT m.rna_type, COUNT(*) AS regions, AVG(r.bit_score) AS avg_bit_score
FROM rfam.full_region r                                      -- MySQL, 10.6M rows
JOIN rnacentral.rfam_models m ON m.rfam_model_id = r.rfam_acc -- PostgreSQL
JOIN mongo.family_notes n ON n.rfam_acc = r.rfam_acc          -- MongoDB Atlas
WHERE n.curation_status = 'reviewed'
GROUP BY m.rna_type ORDER BY regions DESC LIMIT 8
```

Argus processes this query in six steps:

1. The parser resolves `rfam.full_region` to MySQL, `rnacentral.rfam_models` to PostgreSQL and `mongo.family_notes` to MongoDB Atlas.
2. Because the tables span three systems, the optimiser plans a federated join; from MongoDB it needs only `rfam_acc` and `curation_status`, and the filter on `curation_status` is pushed into a `$match` stage.
3. The ML optimizer predicts 6.5 s for the exact plan and 25 ms for the estimate, and chooses the estimate.
4. PAFS selects the largest table, `full_region`, and reads its stored 1% sample of 107,219 rows; the two smaller tables come from local copies.
5. The sample is joined with the other tables, and Horvitz-Thompson estimates with 95% intervals are computed for every RNA type.
6. The answer is returned in 28 ms, against 241 s for the exact query.

Table 6. Exact and estimated results of the three-database query

RNA type	Regions (exact)	Regions (estimate)	Error	Mean bit score (exact)	Mean bit score (estimate)	Error
tRNA	5,336,057	5,320,400	−0.29%	44.08	43.99	−0.22%
other	878,829	870,100	−0.99%	77.84	78.29	+0.58%
rRNA	573,045	564,800	−1.44%	760.45	761.81	+0.18%
snoRNA	492,135	494,900	+0.56%	64.15	64.25	+0.15%
precursor_RNA	426,998	420,500	−1.52%	83.52	83.85	+0.40%

RNA type	Regions (exact)	Regions (estimate)	Error	Mean bit score (exact)	Mean bit score (estimate)	Error
SRP_RNA	423,250	435,400	+2.87%	85.41	85.84	+0.50%
hammerhead_ribozyme	326,880	328,000	+0.34%	44.62	44.88	+0.59%
snRNA	197,103	194,100	−1.52%	96.97	95.26	−1.76%

The mean error over the 16 values was 0.87%, the largest 2.87%, and every exact value fell inside its 95% interval. The ranking of the RNA types is the same in both answers apart from two neighbours whose counts differ by less than their interval widths, so the estimate leads to exactly the same conclusions as the exact answer, four minutes earlier. Figure 6 shows the result as it appears in the workbench.

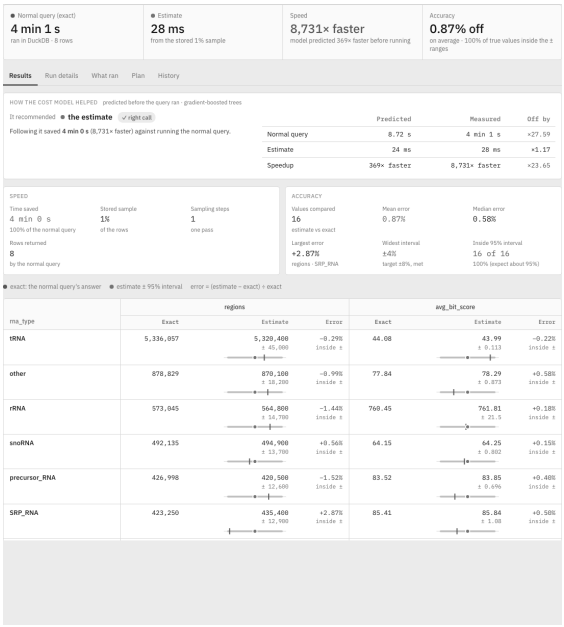


Figure 6. Results screen for the three-database query: 4 min 1 s for the exact query against 28 ms for the estimate (8,731× faster), with the error of every value and the ML optimizer's verdict

5.4 ML optimizer in action

In Figure 5 the estimate was forced on purpose. In normal use the ML optimizer first predicts both running times and only the faster strategy is executed. Table 7 shows its decisions on three representative queries.

Table 7. Decisions made in Auto mode

Query	Data location	Predicted exact / estimate	Model's choice	Time in Auto mode
Group by over 1M-row `lineitem`	Local DuckDB	3 ms / 32 ms	Exact	3.3 ms
Aggregate over 10.6M-row `full_region`	Remote MySQL	5.85 s / 4 ms	Estimate	2.0 ms
Three-database join (Section 5.3)	MySQL, PostgreSQL, MongoDB	6.5 s / 25 ms	Estimate	14 ms

On the local table, where an estimate would have been about ten times slower, the optimizer ran the exact query; on remote data it ran the estimate. Across 317 queries on datasets the model had never seen, its choices compare with other policies as follows.

Table 8. Execution policies on unseen datasets

Policy	Agreement with oracle	Slowdown per query	Total time, 317 queries
Always exact	91%	1.71×	286.08 s
Always approximate	9%	3.00×	28.22 s
Hand-written rule	98%	1.01×	10.18 s
ML optimizer (Auto mode)	97%	1.01×	10.19 s
Oracle (knows the true times)	100%	1.00×	11.35 s

The ML optimizer agreed with the oracle on 97% of queries, needed 28 times less total time than always running the exact query and 2.8 times less than always estimating, and chose an estimate for only 2 of the 291 local queries. It reaches the quality of an expert-tuned rule, but learns it automatically from logged runs, so it adapts as new sources and workloads are added.

5.5 Accuracy and reliability

Across 30 repeated runs per benchmark query, 93% to 100% of exact values fell inside their 95% confidence intervals, which confirms that the reported error bounds can be trusted. The Fast profile reached ±17% with about 9% of the rows and the Balanced profile ±7% with about 37%, so even a small sample is enough for a reliable answer. On the large remote MySQL and PostgreSQL tables the mean error was mostly below 2%, well inside the requested margin.

6. CONCLUSION AND FUTURE SCOPE

Argus demonstrates that one SQL query can join MySQL, PostgreSQL and MongoDB and still return in milliseconds, with an error the user can see and verify. On a join of all three systems over 10.6 million rows, the PAFS estimate took 28 ms against 241 s for the exact query, a speedup of about 8,700 times, with a mean error of 0.87% and every true value inside its confidence interval.

The project shows three things. First, SQL and NoSQL data can be combined without copying everything into a warehouse, provided the optimiser pushes work into each source and fetches only the fields it needs. Second, for remote data the decisive factor is where the sample lives: a sample kept locally removes the network from every later estimate, which is why PAFS outperforms both federated execution and server-side sampling. Third, an ML-based optimizer can decide reliably, query by query, whether an estimate is worth it, choosing correctly for 97% of queries on unseen data.

The system can be extended in several directions: sketches such as HyperLogLog to approximate COUNT(DISTINCT), stratified samples so that very rare groups are always represented, incremental refresh of stored samples when the source data changes, support for further engines

such as Oracle or Cassandra, and continued training of the optimizer on large cross-database workloads.

REFERENCES

[1] J. M. Hellerstein, P. J. Haas and H. J. Wang, "Online aggregation," in Proc. ACM SIGMOD Int. Conf. on Management of Data, 1997, pp. 171–182.

[2] P. J. Haas and J. M. Hellerstein, "Ripple joins for online aggregation," in Proc. ACM SIGMOD, 1999, pp. 287–298.

[3] S. Acharya, P. B. Gibbons, V. Poosala and S. Ramaswamy, "The Aqua approximate query answering system," in Proc. ACM SIGMOD, 1999, pp. 574–576.

[4] S. Acharya, P. B. Gibbons, V. Poosala and S. Ramaswamy, "Join synopses for approximate query answering," in Proc. ACM SIGMOD, 1999, doi: 10.1145/304182.304207.

[5] S. Agarwal et al., "BlinkDB: Queries with bounded errors and bounded response times on very large data," in Proc. 8th ACM European Conf. on Computer Systems (EuroSys), 2013, pp. 29–42.

[6] S. Kandula et al., "Quickr: Lazily approximating complex ad-hoc queries in big data clusters," in Proc. ACM SIGMOD, 2016, pp. 631–646.

[7] Y. Park, B. Mozafari, J. Sorenson and J. Wang, "VerdictDB: Universalizing approximate query processing," in Proc. ACM SIGMOD, 2018, pp. 1461–1476.

[8] S. Chaudhuri, B. Ding and S. Kandula, "Approximate query processing: No silver bullet," in Proc. ACM SIGMOD, 2017, pp. 511–519.

[9] P. Flajolet, É. Fusy, O. Gandouet and F. Meunier, "HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm," in Proc. Conf. on Analysis of Algorithms (AofA), 2007.

[10] G. Cormode, M. Garofalakis, P. J. Haas and C. Jermaine, "Synopses for massive data: Samples, histograms, wavelets, sketches," Foundations and Trends in Databases, vol. 4, no. 1–3, pp. 1–294, 2012.

[11] D. G. Horvitz and D. J. Thompson, "A generalization of sampling without replacement from a finite universe," Journal of the American Statistical Association, vol. 47, no. 260, pp. 663–685, 1952.

[12] C.-E. Särndal, B. Swensson and J. Wretman, Model Assisted Survey Sampling. New York: Springer, 1992.

[13] W. G. Cochran, Sampling Techniques, 3rd ed. New York: Wiley, 1977.

[14] F. Olken and D. Rotem, "Random sampling from databases: A survey," Statistics and Computing, vol. 5, no. 1, pp. 25–42, 1995.

[15] P. J. Haas and C. König, "A bi-level Bernoulli scheme for database sampling," in Proc. ACM SIGMOD, 2004, pp. 275–286.

[16] S. Chaudhuri, R. Motwani and V. Narasayya, "On random sampling over joins," in Proc. ACM SIGMOD, 1999, pp. 263–274.

[17] F. Li, B. Wu, K. Yi and Z. Zhao, "Wander join: Online aggregation via random walks," in Proc. ACM SIGMOD, 2016.

[18] L. M. Haas, D. Kossmann, E. L. Wimmers and J. Yang, "Optimizing queries across diverse data sources," in Proc. 23rd Int. Conf. on Very Large Data Bases (VLDB), 1997, pp. 276–285.

[19] J. Duggan et al., "The BigDAWG polystore system," ACM SIGMOD Record, vol. 44, no. 2, pp. 11–16, 2015.

[20] R. Sethi et al., "Presto: SQL on everything," in Proc. IEEE 35th Int. Conf. on Data Engineering (ICDE), 2019, pp. 1802–1813.

[21] E. Begoli et al., "Apache Calcite: A foundational framework for optimized query processing over heterogeneous data sources," in Proc. ACM SIGMOD, 2018, pp. 221–230.

[22] M. Raasveldt and H. Mühleisen, "DuckDB: An embeddable analytical database," in Proc. ACM SIGMOD, 2019, pp. 1981–1984.

[23] A. Ganapathi et al., "Predicting multiple metrics for queries: Better decisions enabled by machine learning," in Proc. IEEE 25th Int. Conf. on Data Engineering (ICDE), 2009, pp. 592–603.

[24] M. Akdere et al., "Learning-based query performance modeling and prediction," in Proc. IEEE 28th Int. Conf. on Data Engineering (ICDE), 2012, pp. 390–401.

[25] R. Marcus and O. Papaemmanouil, "Plan-structured deep neural network models for query performance prediction," Proceedings of the VLDB Endowment, vol. 12, no. 11, pp. 1733–1746, 2019.

[26] J. Sun and G. Li, "An end-to-end learning-based cost estimator," Proceedings of the VLDB Endowment, vol. 13, no. 3, pp. 307–319, 2019.

[27] A. Kipf et al., "Learned cardinalities: Estimating correlated joins with deep learning," in Proc. Conf. on Innovative Data Systems Research (CIDR), 2019.

[28] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[29] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[30] T. Mao, "sqlglot: Python SQL parser and transpiler." Available: https://github.com/tobymao/sqlglot

[31] I. Kalvari et al., "Rfam 14: Expanded coverage of metagenomic, viral and microRNA families," Nucleic Acids Research, vol. 49, no. D1, pp. D192–D200, 2021.

[32] The RNAcentral Consortium, "RNAcentral 2021: Secondary structure integration, improved sequence search and new member databases," Nucleic Acids Research, vol. 49, no. D1, pp. D212–D220, 2021.

[33] Transaction Processing Performance Council, "TPC Benchmark H: Standard specification." Available: https://www.tpc.org/tpch/

[34] MongoDB, Inc., "Sample Mflix dataset," MongoDB Atlas documentation. Available: https://www.mongodb.com/docs/atlas/sample-data/sample-mflix/